

amazon Logistics Intelligence

Amazon Logistics & Supply Chain — Data Analytics Division

INTERNAL COMMUNICATION — AMAZON DATA ANALYTICS TEAM

From: Priya Menon, Senior Business Analyst — Amazon Logistics Intelligence

To: Data Science & Analytics Team, Amazon India Operations

CC: Rahul Sharma (ML Engineering), Sneha Iyer (Data Engineering)

Date: April 08, 2026

Subject: Data Preprocessing & Feature Engineering Tasks — Amazon Delivery Analytics Dataset

Dear Data Science & Analytics Team,

Greetings from the Logistics Intelligence division.

We are currently working on enhancing our Last-Mile Delivery Analytics system at Amazon India. As part of our quarterly performance review initiative, we need to develop a robust data preprocessing and feature engineering pipeline using our delivery operations dataset.

We have attached the primary delivery dataset (CSV) along with the delivery partner (agent) profile data (JSON). Your team is requested to perform a complete preprocessing and feature engineering workflow to make the dataset model-ready for our predictive delivery time and customer satisfaction scoring systems.

Detailed task requirements are outlined below.

Project Requirements Document

PARAMETER	DETAILS
Project Name	Amazon Last-Mile Delivery Analytics — Preprocessing Pipeline
Client Division	Amazon Logistics Intelligence, India Operations
Dataset (Primary)	amazon_deliveries.csv — 45,000+ delivery records
Dataset (Secondary)	amazon_agents.json — Delivery partner profiles
Problem Type	Regression + Classification (ETA prediction, delay flagging)
Expected Timeline	2 Business Days (Simulation / Academic Project)
Stack	Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn

Dataset Overview

Primary Dataset: amazon_deliveries.csv

Each row represents one delivery order. Key columns include:

COLUMN NAME	DATA TYPE	DESCRIPTION	NOTES
order_id	String	Unique delivery order ID	Format: AMZ-YYYYMMDD-XXXXX
order_datetime	String	Date and time when order was placed	Needs datetime parsing
delivery_city	Categorical	City of delivery (Mumbai, Delhi, Bangalore, etc.)	16 unique cities
delivery_zone	Categorical	Zone within city (A/B/C/D)	Ordinal — A=Premium, D=Remote
distance_km	Float	Distance from warehouse to delivery address	Has ~3% missing values
agent_rating	Float	Delivery agent performance rating (1.0–5.0)	Has ~2% missing values
weather_condition	Categorical	Weather at delivery time	Has ~4% missing values
traffic_index	Float	Traffic congestion score (0–10)	Has ~5% missing values
vehicle_type	Categorical	Type of vehicle used	Bike / Scooter / Van / Cycle
package_weight_kg	Float	Weight of the delivery package	Contains outliers (mis-entries)
delivery_time_min	Float	Actual delivery time in minutes (Target Variable)	Right-skewed distribution

is_delayed	Integer	Binary flag: 1 = delayed, 0 = on time	Derived target for classification
------------	---------	---------------------------------------	-----------------------------------

Secondary Dataset: amazon_agents.json

Each record represents one delivery partner profile:

FIELD	TYPE	DESCRIPTION
agent_id	String	Unique agent identifier
agent_name	String	Full name of the delivery partner
experience_years	Float	Years of experience (has missing values for new agents)
vehicle_owned	String	Type of vehicle owned
city_assigned	String	Primary city of operation
joining_date	String	Date agent joined Amazon Flex program

BUSINESS CONTEXT

Amazon India processes over 8 million deliveries daily. Even a 1-minute improvement in average delivery time translates to significant cost savings.

This dataset covers 45,000+ deliveries across 16 Indian cities over a 6-month window.

The preprocessing pipeline developed here will feed directly into our ETA Prediction Model and Customer Satisfaction Scoring System.

TASK 1 | Data Understanding & Loading

Task 1: Data Understanding & Loading

Load both datasets and perform a thorough initial inspection before any processing begins.

1.1 Load Datasets

- Load amazon_deliveries.csv using Pandas with appropriate dtype specifications
- Load amazon_agents.json using Python json module and convert to DataFrame
- Merge both datasets on the agent_id / order_agent_id key
- Inspect the merged dataset shape, preview first 10 rows

1.2 Initial Dataset Inspection

- Report total number of rows and columns
- Print data types of every column using df.dtypes
- Identify columns with incorrect data types (e.g., order_datetime stored as object)
- Count unique values in each categorical column (delivery_city, vehicle_type, weather_condition, delivery_zone)
- Compute basic statistics for numerical columns (mean, median, std, min, max)

1.3 Missing Value Inventory

- Display missing value count and percentage for each column
- Flag columns with missingness > 5% for advanced imputation treatment
- Visualize missing value pattern using a heatmap (seaborn)

DELIVERABLE

A consolidated data quality report table showing: column name, dtype, null count, null %, unique values, and recommended action.

TASK 2 | Data Cleaning

Task 2: Data Cleaning

Clean all identified data quality issues before proceeding with analysis or modeling.

2.1 Handle Missing Values (Initial Pass)

- `distance_km` — Apply median imputation (numerical, right-skewed)
- `agent_rating` — Apply mean imputation (approximately normal distribution)
- `weather_condition` — Apply most frequent (mode) imputation (categorical)
- `traffic_index` — Apply median imputation (contains outlier influence)
- `experience_years` (from agents JSON) — Use 0 for new agents; flag with a binary indicator

2.2 Datetime Conversion

- Convert `order_datetime` from string to proper datetime format using `pd.to_datetime()`
- Standardize timezone if applicable (IST — UTC+5:30)
- Validate that all records parse successfully; log unparseable entries

2.3 Duplicate Record Removal

- Identify duplicate rows based on `order_id` (exact duplicates)
- Identify near-duplicates: same `order_datetime` + `delivery_city` + `agent_id`
- Remove all confirmed duplicates and reset DataFrame index
- Report count of records removed

2.4 Fix Inconsistent Values

- Standardize city names: remove extra spaces, fix casing (e.g., "mumbai", "MUMBAI" → "Mumbai")
- Fix `vehicle_type` inconsistencies (e.g., "bike", "Bike", "BIKE" → "Bike")
- Cap `package_weight_kg` at physically plausible maximum (50 kg for standard delivery)
- Flag `delivery_time_min` < 5 minutes as data entry errors and treat accordingly

DELIVERABLE

A cleaned DataFrame saved as `amazon_deliveries_cleaned.csv`.

A cleaning log summarizing: issues found, strategy applied, and records affected for each column.

TASK 3 | Exploratory Data Analysis (EDA)

Task 3: Exploratory Data Analysis

Perform structured EDA at three levels — Univariate, Bivariate, and Multivariate — to extract actionable insights about Amazon delivery operations.

3.1 Univariate Analysis

Analyze each feature individually to understand its distribution and characteristics:

- distance_km — Histogram + Boxplot; compute skewness and kurtosis
- delivery_time_min — Histogram; check for bimodal distribution (city vs rural deliveries)
- agent_rating — Distribution plot; check normality assumption
- package_weight_kg — Boxplot to identify outliers visually
- traffic_index — Bar chart of binned traffic levels (Low / Medium / High / Severe)
- delivery_city — Bar chart of top 10 cities by volume
- vehicle_type — Pie chart of vehicle distribution
- is_delayed — Count plot; document class imbalance ratio

3.2 Bivariate Analysis

Examine relationships between key feature pairs:

- distance_km vs delivery_time_min — Scatter plot with regression line; compute Pearson correlation
- traffic_index vs delivery_time_min — Scatter plot; higher traffic should increase delivery time
- vehicle_type vs delivery_time_min — Box plots per vehicle type; identify fastest vehicle category
- delivery_city vs is_delayed — Grouped bar chart; identify cities with highest delay rates
- weather_condition vs delivery_time_min — Violin plots; check impact of rain / fog on ETA
- agent_rating vs is_delayed — Bar plot of average delay rate by rating band

3.3 Multivariate Analysis

Analyze combined effects of multiple features on delivery outcomes:

- Correlation heatmap: all numerical features vs delivery_time_min
- Pivot table: delivery_city x traffic_index (binned) → mean delivery_time_min
- Grouped analysis: city + vehicle_type + weather_condition → average delay %
- Pair plot: distance_km, delivery_time_min, agent_rating, traffic_index coloured by is_delayed
- Generate an automated profiling report using ydata-profiling

3.4 Key Insights to Document

- Top 3 features most correlated with delivery_time_min
- Cities with delay rate above the national average
- Whether weekend/night deliveries take longer than weekday/daytime
- Distribution of delivery times by vehicle type and zone

DELIVERABLE

An EDA notebook section with all plots properly labelled and annotated.

A written "Key Insights" block of at least 8 observations derived from the analysis.

TASK 4 | Advanced Missing Value Treatment

Task 4: Advanced Missing Value Treatment

Go beyond basic imputation. Apply multiple techniques and compare their effectiveness on the Amazon delivery dataset.

4.1 Simple Imputer

- Apply mean strategy for agent_rating
- Apply median strategy for distance_km and traffic_index
- Apply most_frequent strategy for weather_condition and vehicle_type
- Document imputed values chosen by each strategy

4.2 Most Frequent / Category Imputation

- For weather_condition: use mode imputation as a baseline
- For delivery_zone: use most frequent zone per city (grouped mode)
- Create a binary Missing Indicator feature for experience_years to preserve missingness information

4.3 KNN Imputer

- Apply KNNImputer (k=5) to numerical features: distance_km, agent_rating, traffic_index, package_weight_kg
- Justify choice of k using cross-validation or domain knowledge
- Compare imputed values from KNN vs SimpleImputer (median) for each column

4.4 Technique Comparison

Create a comparison table:

COLUMN	SIMPLEIMPUTER (MEDIAN)	KNN IMPUTER (K=5)	RECOMMENDED STRATEGY
distance_km	Fill with global median	Fill using 5 nearest orders	KNN (correlated with city)
agent_rating	Fill with mean rating	Fill using similar agents	KNN (agent-specific)
traffic_index	Fill with median	Fill using location + time context	KNN (time-correlated)
weather_condition	Fill with mode	N/A (categorical)	Most Frequent by City
experience_years	Fill with median	Fill using agent clusters	Median + Missing Indicator

DELIVERABLE

Side-by-side distribution plots (before imputation, after SimpleImputer, after KNN) for each numerical column.

A written recommendation on which imputation strategy to use for the final pipeline and why.

TASK 5 | Outlier Detection & Handling

Task 5: Outlier Detection & Handling

Delivery datasets frequently contain outliers due to data entry errors, exceptional orders (bulk deliveries), or system anomalies. Detect and handle them systematically.

5.1 Z-Score Method

- Compute Z-scores for: distance_km, delivery_time_min, package_weight_kg, traffic_index
- Flag records with $|Z| > 3$ as outliers
- Report count and percentage of outliers detected per column
- Visualize using Z-score distribution plot with threshold lines

5.2 IQR Method

- Compute Q1, Q3, and IQR for the same numerical columns
- Flag values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$
- Compare with Z-score method: do they agree on the same records?
- Plot box plots before and after IQR-based removal

5.3 Percentile Method

- Cap distance_km at the 1st and 99th percentile
- Cap delivery_time_min at the 2nd and 98th percentile
- Cap package_weight_kg at the 99th percentile (no lower cap — can be very small)
- Visualize the capped distributions to confirm effect

5.4 Winsorization

- Apply `scipy.stats.mstats.winsorize` with `limits=[0.02, 0.02]` to package_weight_kg
- Apply Winsorization with `limits=[0.01, 0.05]` to delivery_time_min (preserve short deliveries)
- Compare Winsorized distribution to original using overlay histogram

5.5 Impact Analysis

Document before-and-after statistics for each treated column:

COLUMN	BEFORE TREATMENT (MEAN / STD)	AFTER TREATMENT (MEAN / STD)	RECORDS AFFECTED
distance_km	___ / ___	___ / ___	___
delivery_time_min	___ / ___	___ / ___	___
package_weight_kg	___ / ___	___ / ___	___
traffic_index	___ / ___	___ / ___	___

DELIVERABLE

Overlay plots (before vs after treatment) for all four numerical columns.

A written recommendation: which method is most appropriate for the Amazon delivery context and why.

TASK 6 | Feature Engineering

Task 6: Feature Engineering

Create business-meaningful features from raw data to improve predictive model performance.

6.1 Datetime Feature Extraction from order_datetime

- order_hour — Hour of the day (0–23)
- order_day_of_week — Day name (Monday–Sunday)
- order_day — Day of the month (1–31)
- order_month — Month number (1–12)
- order_quarter — Quarter (Q1–Q4)
- is_weekend — Binary flag: 1 if Saturday or Sunday

6.2 Business-Specific New Features

- is_peak_hour — 1 if order_hour is in [9, 10, 11, 18, 19, 20, 21] (Amazon rush hours)
- is_night_delivery — 1 if order_hour is in [22, 23, 0, 1, 2, 3, 4, 5]
- distance_category — Categorical: Short (< 5 km), Medium (5–15 km), Long (> 15 km)
- traffic_severity — Categorical from traffic_index: Low (0–3), Moderate (3–6), High (6–8), Severe (8–10)
- agent_experience_band — New / Junior / Senior based on experience_years thresholds
- weight_category — Light (< 1 kg), Standard (1–5 kg), Heavy (> 5 kg)

6.3 Numerical Transformations

- Log transformation — Apply to delivery_time_min and distance_km (right-skewed)
- Square root transformation — Apply to package_weight_kg
- Compare skewness before and after transformation for each column
- Plot distribution before and after transformation side by side

DELIVERABLE

Updated DataFrame with all new features added.

A feature summary table: feature name, creation method, data type, and business rationale.

Skewness comparison table before and after transformations.

TASK 7 | Handling Mixed & Special Variables

Task 7: Handling Mixed & Special Variables

7.1 Separate Feature Types

- Identify and list all numerical features (continuous + discrete)
- Identify and list all categorical features (nominal + ordinal)
- Identify all datetime-derived features
- Identify binary features (already 0/1 encoded)
- Print a feature type inventory table

7.2 Handling datetime Features

- Confirm order_datetime and joining_date (from agents) are properly converted to datetime64
- Extract agent_tenure_days = (reference_date - joining_date).days
- Drop raw datetime columns after all features are extracted

7.3 Handling order_id (High-Cardinality ID Column)

- Extract date component from order_id format AMZ-YYYYMMDD-XXXXX
- Drop order_id after extraction — ID columns should never be fed into models directly

7.4 Final Pre-Encoding Dataset Validation

- Confirm no raw string/object columns remain except those awaiting encoding
- Verify no datetime columns remain as objects
- Print df.info() and confirm all dtypes are numeric or categorical

DELIVERABLE

A clean "pre-encoding" DataFrame with every column correctly typed and no raw mixed or datetime variables remaining.

TASK 8 | Encoding Categorical Data

Task 8: Encoding Categorical Features

Convert all categorical variables into numerical format suitable for machine learning algorithms.

8.1 Label Encoding

- Apply LabelEncoder to binary columns: is_delayed, is_weekend, is_peak_hour
- Apply LabelEncoder to vehicle_type as baseline for comparison
- Note: Label encoding implies ordering — acceptable for binary features, risky for multi-class nominal features

8.2 One-Hot Encoding

- Apply OneHotEncoder (drop="first") to: delivery_city, vehicle_type, weather_condition
- Justify drop="first" to avoid dummy variable trap and multicollinearity
- Document column explosion: how many new columns are created per feature?
- Check for rare categories with < 1% frequency and group them into "Other"

8.3 Ordinal Encoding

- Apply OrdinalEncoder to delivery_zone: D=0, C=1, B=2, A=3 (D=Remote, A=Premium urban)
- Apply OrdinalEncoder to traffic_severity: Low=0, Moderate=1, High=2, Severe=3
- Apply OrdinalEncoder to distance_category: Short=0, Medium=1, Long=2
- Apply OrdinalEncoder to agent_experience_band: New=0, Junior=1, Senior=2

8.4 Encoding Comparison Summary

FEATURE	ENCODING METHOD	RATIONALE
delivery_city	One-Hot Encoding	Nominal — no ordering between cities
vehicle_type	One-Hot Encoding	Nominal — no ordering between vehicle types
weather_condition	One-Hot Encoding	Nominal — weather types are not ordered
delivery_zone	Ordinal Encoding	Ordinal — A > B > C > D by service quality
traffic_severity	Ordinal Encoding	Ordinal — Low < Moderate < High < Severe
distance_category	Ordinal Encoding	Ordinal — Short < Medium < Long
agent_experience_band	Ordinal Encoding	Ordinal — New < Junior < Senior
is_delayed, is_weekend	Label Encoding	Binary — no ordering issue

DELIVERABLE

Encoded DataFrame with all categorical columns transformed.
Before/after column count comparison.

TASK 9 | Numerical Feature Transformation

Task 9: Numerical Feature Binning & Discretization

Transform continuous numerical features into discrete bins or binary flags to capture non-linear relationships.

9.1 Binning (Equal-Width Discretization)

- Apply KBinsDiscretizer (strategy="uniform", n_bins=5) to distance_km
- Apply KBinsDiscretizer (strategy="uniform", n_bins=4) to delivery_time_min
- Label bins meaningfully: Very Short, Short, Medium, Long, Very Long

9.2 Binarization

- Create is_long_distance = 1 if distance_km > 15 km, else 0
- Create is_slow_delivery = 1 if delivery_time_min > 60 minutes, else 0
- Create is_heavy_package = 1 if package_weight_kg > 5 kg, else 0

9.3 Quantile Binning (Equal-Frequency)

- Apply KBinsDiscretizer (strategy="quantile", n_bins=4) to agent_rating
- Apply KBinsDiscretizer (strategy="quantile", n_bins=5) to traffic_index
- Compare bin boundaries vs equal-width: which creates more balanced bins?

9.4 K-Means Binning

- Apply KBinsDiscretizer (strategy="kmeans", n_bins=4) to delivery_time_min
- Cluster-based bins reflect natural groupings in delivery time patterns
- Compare cluster centroids with percentile-based boundaries

DELIVERABLE

New binned/binarized feature columns added to the DataFrame.
Distribution plots for original vs binned features for each column.
A comparison table: bin boundaries across Uniform, Quantile, and K-Means strategies.

TASK 10 | Feature Scaling

Task 10: Feature Scaling

Apply and compare different scaling techniques to ensure all features contribute appropriately to the model.

Apply each of the following scalers to the numerical feature set: [distance_km, delivery_time_min, agent_rating, traffic_index, package_weight_kg, agent_tenure_days]

10.1 Standardization (StandardScaler)

- Scale each feature to have mean = 0 and standard deviation = 1
- Formula: $z = (x - \mu) / \sigma$
- Best for: Linear Regression, Logistic Regression, SVM, PCA

10.2 Min-Max Normalization (MinMaxScaler)

- Scale features to range [0, 1]
- Formula: $x_{\text{norm}} = (x - \min) / (\max - \min)$
- Best for: Neural Networks, KNN, algorithms sensitive to feature magnitude
- Caution: Sensitive to outliers — apply after outlier treatment

10.3 MaxAbs Scaling (MaxAbsScaler)

- Scale to range [-1, 1] by dividing by maximum absolute value
- Preserves sparsity — suitable for one-hot encoded matrices
- Apply to the full OHE-encoded feature matrix

10.4 Robust Scaling (RobustScaler)

- Scale using median and IQR: $z = (x - Q2) / \text{IQR}$
- Robust to outliers — recommended for delivery_time_min and distance_km
- Best for: Amazon delivery data with known outliers in billing and distance

10.5 Comparison of Scaling Effects

SCALER	OUTPUT RANGE	OUTLIER SENSITIVITY	RECOMMENDED FOR
StandardScaler	Unbounded, $\sim N(0,1)$	Moderate	Logistic Regression, SVM, PCA
MinMaxScaler	[0, 1]	High	Neural Networks, KNN
MaxAbsScaler	[-1, 1]	High	Sparse OHE matrices

RobustScaler	Unbounded (IQR)	Low	delivery_time_min, distance_km
--------------	-----------------	-----	--------------------------------

DELIVERABLE

Side-by-side distribution plots for all 4 scalers applied to delivery_time_min.

A written recommendation on which scaler to use in the final Amazon delivery pipeline.

Critical rule demonstrated: fit on training set only, transform on train + test.

TASK 11 | Advanced Feature Transformation & Pipeline

Task 11: Advanced Feature Transformation & ColumnTransformer Pipeline

Apply power and custom function transformations, then consolidate all preprocessing into a single, production-ready ColumnTransformer pipeline.

11.1 FunctionTransformer

- Log Transform (`np.log1p`) — Apply to `delivery_time_min` and `distance_km` (right-skewed, positive)
- Reciprocal Transform (`1/x`) — Apply to `traffic_index` as `1/traffic_index` (model free-flow as high value)
- Square Root Transform (`np.sqrt`) — Apply to `package_weight_kg`
- For each: plot before vs after distribution and compute skewness delta

11.2 PowerTransformer

- Box-Cox Transform — Apply to `delivery_time_min` (strictly positive values)
- Box-Cox automatically finds optimal λ (lambda) — report the fitted lambda value
- Yeo-Johnson Transform — Apply to `traffic_index` (handles zero values)
- Compare: which transform achieves skewness closest to 0 for each feature?

11.3 Final ColumnTransformer Pipeline

Assemble all preprocessing steps into a unified, modular sklearn Pipeline:

PIPELINE STEP	TRANSFORMER	APPLIED TO
Numerical Imputation	SimpleImputer (median)	distance_km, delivery_time_min, agent_rating, traffic_index, package_weight_kg
Power Transform	PowerTransformer (yeo-johnson)	distance_km, delivery_time_min, package_weight_kg
Scaling	RobustScaler	All numerical features
Categorical Imputation	SimpleImputer (most_frequent)	weather_condition, vehicle_type, delivery_zone
Ordinal Encoding	OrdinalEncoder (custom category order)	delivery_zone, traffic_severity, distance_category
One-Hot Encoding	OneHotEncoder (drop=first)	delivery_city, vehicle_type, weather_condition

DELIVERABLE

A fully functional sklearn ColumnTransformer + Pipeline object that can be called with `pipeline.fit_transform(X_train)` and `pipeline.transform(X_test)`.
Skewness comparison table: before vs after each PowerTransformer method.

Confirmation that the pipeline has no data leakage: all fit() calls on training data only.

Expected Deliverables

Primary Deliverables

- Jupyter Notebook (.ipynb) — Complete preprocessing workflow with all 11 tasks, clean code, markdown explanations, and visualizations for each step
- Cleaned Dataset — amazon_deliveries_processed.csv ready for downstream ML modeling
- Preprocessing Pipeline Script — pipeline.py containing the final ColumnTransformer pipeline as a reusable Python module
- EDA Report — Auto-generated profiling report (HTML) from ydata-profiling

Documentation Requirements

- Each task must include a markdown cell with: objective, approach, observations, and business interpretation
- All plots must have titles, axis labels, and legends
- A final Summary section at the end of the notebook listing: total records cleaned, features engineered, recommended preprocessing strategy, and top 5 EDA insights

Evaluation Criteria

CRITERION	WEIGHT	DESCRIPTION
Data Cleaning Effectiveness	20%	Completeness and correctness of null handling, deduplication, and inconsistency fixing
EDA Depth & Insights	20%	Quality of visualizations, business-relevant observations, and anomaly identification
Preprocessing Technique Accuracy	25%	Correct application and justification of each imputation, outlier, encoding, scaling technique
Feature Engineering Quality	15%	Relevance of new features, datetime extraction completeness, transformation effectiveness
Pipeline Design	10%	Modularity of ColumnTransformer, no data leakage, clean train/test separation
Code Readability	10%	Function usage, comments, variable naming, modular structure

Technical Guidelines

Required Python Libraries

LIBRARY	VERSION (MINIMUM)	PURPOSE
pandas	2.0+	Data loading, cleaning, transformation
numpy	1.24+	Numerical operations and array manipulation
scikit-learn	1.3+	Imputers, encoders, scalers, transformers, Pipeline
matplotlib	3.7+	Base plotting
seaborn	0.12+	Statistical visualizations
scipy	1.11+	Z-score, Winsorization
ydata-profiling	4.0+	Automated EDA report generation

Code Quality Standards

- Use functions for any operation repeated more than twice (DRY principle)
- Name variables descriptively: df_cleaned, df_encoded, X_train_scaled
- Place all imports at the top of the notebook in a dedicated "Setup" cell
- Add a markdown header cell before each of the 11 Tasks
- Seed all random operations with random_state=42 for reproducibility
- Use assert statements to validate data shape and null counts after each step

Submission Format

- Notebook file: amazon_delivery_preprocessing.ipynb
- Cleaned data: amazon_deliveries_processed.csv
- Pipeline script: amazon_preprocessing_pipeline.py
- EDA report: amazon_eda_report.html
- Submit all files in a single ZIP: amazon_ds_project_<your_name>.zip

Project Timeline

DAY	SESSION	TASKS	EXPECTED OUTPUT
Day 1	Morning (3 hrs)	Tasks 1–3: Data Loading, Cleaning, EDA	Cleaned notebook, EDA report
Day 1	Afternoon (3 hrs)	Tasks 4–5: Advanced Imputation, Outlier Treatment	Imputation comparison plots
Day 2	Morning (3 hrs)	Tasks 6–8: Feature Engineering, Mixed Vars, Encoding	Encoded feature DataFrame
Day 2	Afternoon (3 hrs)	Tasks 9–11: Binning, Scaling, Pipeline Assembly	Final pipeline + cleaned CSV

IMPORTANT NOTE

This is a simulation project using a synthetic dataset modelled after Amazon India logistics data patterns.

Do not use real Amazon production data. All insights and patterns observed are for academic learning purposes.

The preprocessing pipeline built here follows industry standards used by Amazon's Data Science teams.

Attachments

- amazon_deliveries.csv — Primary delivery orders dataset (45,000+ records, 12 columns)
- amazon_agents.json — Delivery partner profiles (5,000+ records, 6 fields)

Please reach out to the Logistics Intelligence team for any clarifications regarding dataset fields or business context.

We look forward to your comprehensive preprocessing workflow and insights.

Best Regards,

Priya Menon

Senior Business Analyst — Logistics Intelligence

Amazon India Operations | Data Analytics Division

Email: priya.menon@amazon.in | Slack: [@priya.menon](#)